Pixel-to-Meaning on the Operator Host: Multimodal Screen Understanding for Host Co-Pilots

A Full-Length Research Paper

Field / Value

Title / Pixe

Abstract

Large language models excel at language but are blind to the operator's desktop unless an external perception stack is attached. We present the **POCKET Pixel Translator**, a host-local multimodal perception system that converts screen pixels and accessibility structure into agent-actionable meaning. Unlike OCR-only pipelines, the translator fuses three modalities-(1) **semantic UI text** from Windows UI Automation, (2) **optical character recognition** via Windows.Media.Ocr when available, and (3) **pure visual structure** (edge density, regional saliency, palette, mood)-and selects a **primary modality** with an explicit rationale. Empirical live capture on an operator workstation shows 200 accessibility-named controls and 37-53 OCR lines simultaneously, with the fusion policy correctly preferring semantic UI text for application chrome. The system is exposed as a first-class platform API (`/v1/vision/understand`, `/v1/pixel/text`), integrated into observe bridges and skill orchestration, enabling outer agents (Grok Build, Codex, phone clients) to *see* the glass and act through the same POCKET API. We argue this multimodal host perception is a fundable wedge relative to chat-only AI: it operationalizes signed-in browsers, real UI navigation, and commercial demo recording without requiring cloud screen streaming.

1. Introduction

1.1 Motivation

When an operator says "what do you see on my screen?", a cloud chat model has no native answer. When they say "click the third link," the model cannot resolve coordinates without a host sensory layer. Prior POCKET work established desktop allowlists, multi-step doers, browser mode, Latin workers, and orchestrated skills. Missing was a **principled, multimodal translator from pixels (and OS structure) to meaning**, with honesty about when text is *not* optimal.

1.2 Contributions

1. **Three-modality fusion** for host screen understanding: semantic UI, OCR, pure visual.

2. **Primary

1.3 Scope and non-claims

We do **not** claim perfect OCR on every font. We do **not** claim end-to-end purchase automation. We **do** claim a production-usable sensory stack that prefers the right modality and exposes it over HTTP.

2. Related Work and Positioning

Approach / Limitation

Chat LLM alo

POCKET's translator sits **on the operator host**, fuses modalities, and feeds **workers and live bridges** that already control Edge, Copilot, Outlook, and desktop apps under allowlists.

3. System Architecture

+----------------------+

3.1 Capture

Full primary display capture via PIL `ImageGrab`, optional downscale (default max width 1280) for latency and token-friendly previews.

3.2 Modality A - Semantic UI text

Windows UI Automation enumerates on-screen named elements (Name, ControlType, bounding rectangle). This often **dominates OCR** for app chrome: Start, taskbar pins, "Pull requests," Copilot, etc.

3.3 Modality B - OCR

- Prefer **Windows.Media.Ocr** via PowerShell/WinRT when present.

- Optional **

3.4 Modality C - Pure visual

Without reading glyphs:

- Mean RGB, brightness, contrast

- 3x3 region

Busy region centers become **click_xy** candidates when text fails.

3.5 Fusion policy

if semantic_count >= 15:

primary =

3.6 Action hints

- Links/buttons -> `click_name`

- Pure visua

4. Platform API Surface

Method / Path / Role

GET/POST / `

Auth: Basic / `X-Pocket-Access` / `Bearer sk_pocket_...`.

Invariant: Outer agents (including Grok Build) should call these endpoints rather than private offline imports, so the public/platform path stays the only production path.

5. Integration with Workers and Bridges

5.1 Dynamic workers

Goal-conditioned loops call `observe()` each step, read `ui_names` / `action_hints`, then scroll, click, or screenshot. Memory persists under `~/.pocket/worker_brains/`.

5.2 Real-time bridge

Synchronous bridge:

1. `POST /v1/bridge/open` (optional record)

2. `POST /v

This enables *human-in-the-loop or LLM-in-the-loop* control without prewritten multi-app scripts.

5.3 Orchestrator

Skills `understand`, `pixel_text`, `see_screen` dispatch through the orchestrator so chat workflows can request perception as a first-class step.

6. Empirical Live Observation (Operator Host, 2026-07-27)

A live `understand()` call on the operator machine produced:

Signal / Value	
Primary moda	

OCR plain-text fragments referenced AI Studio / Nexus Agentic IDE UI copy, navigation labels ("Code", "START...CHANNEL"), and chat-like content-consistent with a multi-window builder workstation, not a blank desktop.

Interpretation for the outer agent:

The glass is	
pixels appea	
brief** for do	

This section is deliberately empirical: the paper is grounded in a real host observation, not synthetic screenshots alone.

7. What This Enables That Chat-Only AI Cannot

1. **Signed-in host context** - Edge profiles, X, GitHub Desktop stay local.

2. **Modality

8. Limitations and Future Work

Limitation / Mitigation path	
UIA may emp	

Future: ROI-cropped OCR on busiest regions; focused-window UIA only; VLM caption as fourth modality; remote VM host with identical `/v1/vision/*`.

9. Security and Safety

- Perception is **local**; frames stay under `~/.pocket/vision` unless the operator exposes the API.

- Public tunn

10. Related POCKET Subsystems

- Latin workers (ARCHON, OCULUS, PORTARIUS, ...)

- Orchestrat

11. Conclusion

The POCKET Pixel Translator treats the operator screen as a **multimodal signal**, not a single OCR dump. By fusing semantic accessibility text, OCR, and pure visual structure-and by publishing the result on the platform API-we enable outer agents to answer "what do you see?" with evidence and to choose actions that match the optimal modality. Live host experiments confirm simultaneous richness of UIA and OCR, with correct primary selection for app chrome. This perception layer is foundational to host co-pilots that create commercial value beyond chat: real glass, real signed-in sessions, real recordings, real multi-agent work on the machine people already own.

Acknowledgments

Operator validation on the Medina Tech Labs workstation; continuous product pressure toward API-first, non-scripted workers.

References (systems)

1. POCKET Platform API First (INL-2026-POCKET.API.015)

2. Vision Wo

Appendix A - API Examples

GET /v1/vision/understand

Authorizatio

-> {

"primary_m

GET /v1/pixel/text

-> { "text": "=

Appendix B - Artifact Paths

Artifact / Path

Last understa

**End of pap